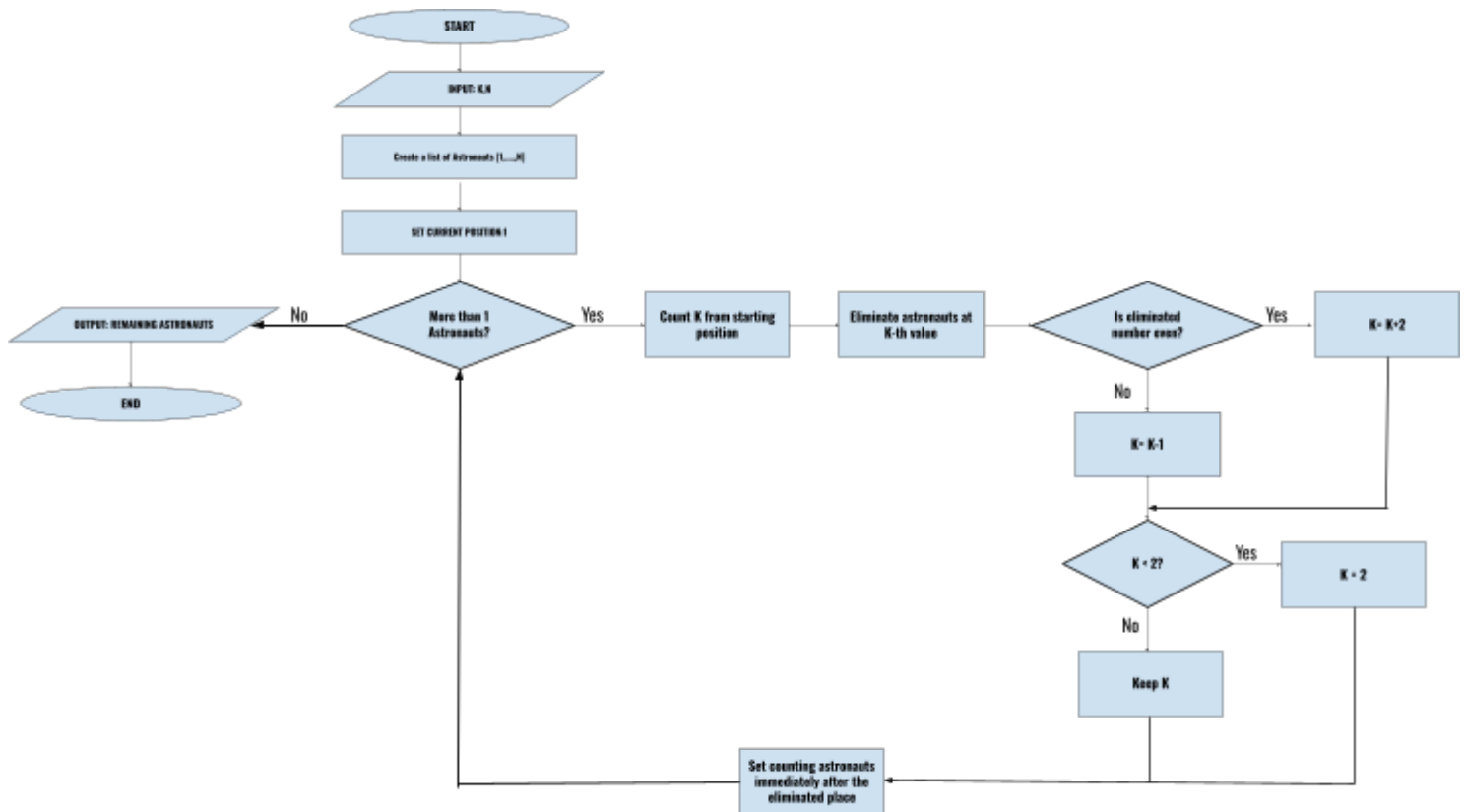


Tugas 1: Flow Chart + Logical explanation



Before the program starts, the program first has to know the number of the astronauts and the initial elimination count. So the first thing to do is to input the N (number of astronauts) and K (the initial elimination count). After inputting the number of K and N, the programs need to set a list of the astronauts from 1 to the designated number so that the astronauts have a unique identifier that is stored in the list, this identifier allows the program to track their position during counting and elimination. Then the program will also set an empty list for initialization elimination order. After setting the unique identifier and empty elimination list, we need to set the current position to 1 so that the program knows that we start from astronauts number 1 which is the beginning. The next step is the program will enter the loop to check and eliminate the astronauts.

The loop starts from count K steps around the circular list. When it reaches the Kth value, the astronauts in the Kth value will be eliminated. Remove the astronauts at the K-th position and record the ID to be put in the elimination order. Then, check the eliminated astronaut's ID whether the ID is odd or even. If the number is even, then K will increase by 2.

But if it is odd, then K will decrease by 1. After that, we need to do another check whether K is less than two or not. If K is greater than 2 then we ask the program to just keep K as it is. But, if K is less than two then we ask the program to set K into 2. After we eliminate the astronauts, set the new starting position to the astronauts immediately following the position of the one just eliminated. The steps will keep repeating until the number of astronauts equals one.

Tugas 2: Pseudocode + Logical explanation

DECLARE

Alphabets : "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
Human_words
Encrypted_messages
value_current
value_prev
shift_value
new_char

BEGIN

GET Human_words
SET Encrypted_messages : ""
SET value_prev = 0

FOR each Char In until end of Human_words
 FOR i from 0 to 25
 IF alphabets [i] == Char
 SET value_current = i + 1
 BREAK
 ENDIF
 ENDIFOR
 SET value_current + value_prev = shift_value
 IF
 Shift_value > 26
 SET shift_value = shift_value - 26
 ENDIF
 SET new_char :alphabet [shift_value - 1]
 SAVE new_char to Encrypted_messages
 SET value_prev = value_current

ENDFOR

PRINT "Alien's language : ", Encrypted_messages

END

First before starting the loop, we need to declare each variable to the right data type so the program can allocate the correct memory type. First, we declare `alphabet`, `Human_words`, `Encrypted_messages` as strings since they will store words/text. Next, `value_current`, `value_prev`, and `shift_value` as integers because they will hold numerical values for the calculation. Last we declare `new_char` as a `Char` because it holds only a single character at a time. We first start the program by getting the words that want to be translated into the Alien word. Then we set the Encrypted message in a blank string to store the output letters as they were later processed in the loop. Lastly, we also set the `value_prev` as 0 because the first letter of the message has no previous letter to shift from.

The loop starts by telling the program to read the inputted character by character and for each character will be converted using a set of systematic way. First the inner loop will repeat the process through the alphabet array from index 0 to 25 to find the index that matches the letter. Once the program finds the correct place, we add index `i` by 1. This is needed because we are converting a zero-based array to the 1-based letter positioning so we need to add by 1. This index of `i+1` will be set as the `value_current` and this marks the end of the inner loop. Next we need to calculate the shift value by adding `value_current` and `value_prev`. After we add we need to check whether the shift value is more or less than 26. If less than 26 then it will continue to just keep counting. But, if more than 26 it needs to be subtracted by 26 so it will go back again to the beginning of the alphabet. The last part of the outer loop is we map the shift value back to a character by assessing the alphabets [`shift_value - 1`], we need to minus it by 1 also because the program start the index from 0, if we don't minus it by 1 then the place of the letter will be wrong. After that we need to stick the new letter to the empty encrypted messages. Before printing the encrypted messages, we need to save the `value_current` into `value_prev` so the current letter's original position serves as the shift for the next character. Last, once the loop ends and finishes converting all characters, the program will print the alien language through the encrypted message.